DaMod — ETL Designer

End-to-End Workflow Handbook

Designing, validating, generating SQL for, and testing a model-driven ETL pipeline in DaMod. Grounded in the current implementation — clearly separating what runs today, what needs an external tool, and documented product gaps.

Version 1.0 · Prepared 2026-10-11

Case study: Retail Sales Analytics Pipeline

Synthetic data only · execution is external

Table of Contents

Table of Contents 2
1. Introduction to DaMod 3
2. ETL concepts and terminology 4
3. DaMod architecture 5
4. Environment setup and prerequisites 6
5. End-to-end business use case 7
6. Source dataset and schema 8
7. Source model creation 9
8. Target model design 10
9. Source-to-target mapping 11
10. Transformation design 12
11. Build the pipeline step by step 13
12. Validation and data quality 14
13. SQL generation and execution 15
14. Verify the final output 16
15. Save, reopen, and maintain pipelines 17
16. Troubleshooting guide 18
17. Test plan and acceptance criteria 19
18. Current limitations and roadmap 20
Appendix A — Quick-start checklist 21
Appendix B — Mapping reference 22
Appendix C — Test data and expected results 23
Appendix D — Glossary and references 24

1. Introduction to DaMod

DaMod is a local-first desktop application for relational and dimensional data modeling, reverse-engineering, and — the subject of this handbook — model-driven ETL design. It stores everything as plain, Git-diffable JSON; there are no accounts and no telemetry.

1.1 What the modeling engine gives you

- Entities, attributes, keys, foreign keys, constraints, and four modeling levels (Conceptual / Logical / Physical-Logical / Physical).
- Reusable **domains**, **Naming Standards**, and **Data Type Standards**.
- Reverse-engineering from SQL DDL, and DDL generation for seven dialects (Postgres, Snowflake, Databricks, Oracle, SQL Server, MySQL, BigQuery).

1.2 How the ETL Designer extends it

The ETL Designer treats your DaMod models as the authoritative source and target schemas. A pipeline references model objects by stable id, then layers a visual transform graph on top: source → filter → select → derive → cast → join → aggregate → distinct → target. From that graph DaMod infers each node's schema, validates the pipeline, and generates SQL and target DDL.

> ■ **NOTE**
> Relationship of concepts: **models/metadata** define schemas; a **pipeline** references them and adds a transform DAG; **validation** checks it; **generation** emits SQL + DDL. Execution is a separate concern (external today).

1.3 Who this is for

Data engineers, analytics engineers, product managers, and QA. The handbook is a practical implementation + testing guide: follow it to build the Retail Sales pipeline, validate it, generate SQL, execute it externally, and verify results against a hand-computed oracle.

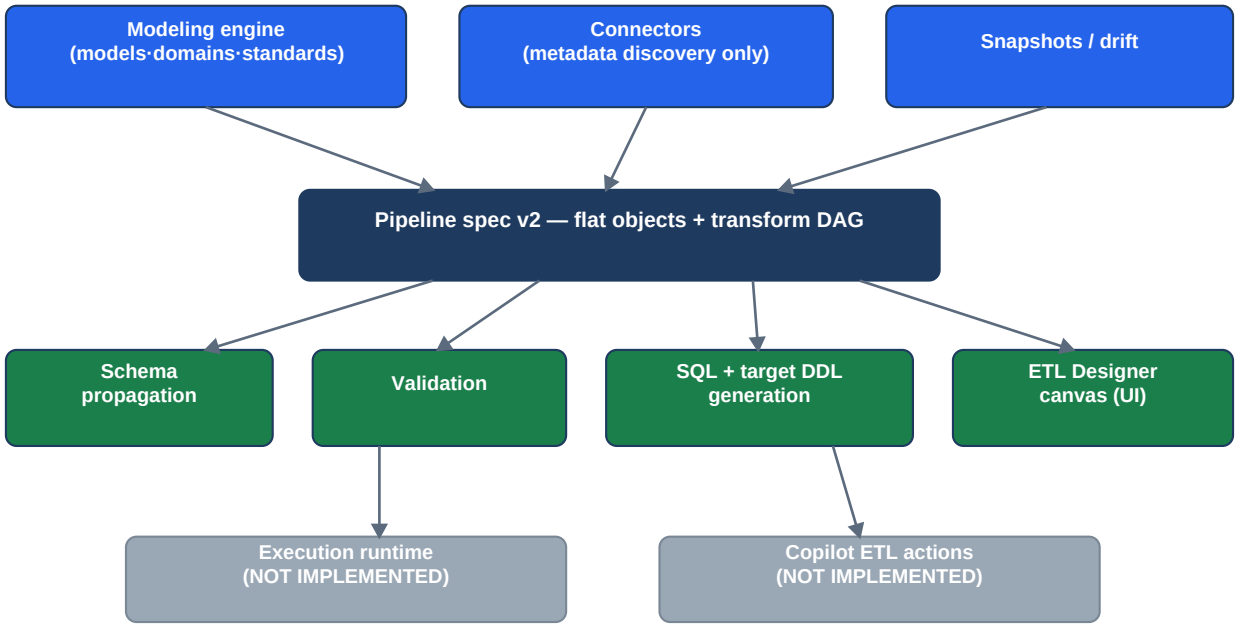
2. ETL concepts and terminology

Term	Practical meaning in DaMod
ETL vs ELT	ETL transforms before load; ELT loads then transforms in the warehouse. DaMod generates SELECT-based SQL you can run either way.
Source / target system	Source = where data originates (a modeled schema); target = the analytics schema you populate.
Metadata discovery	Reading a schema's structure (tables/columns/keys) — NOT its rows. DaMod connectors are discovery-only.
Source / target model	A DaMod model used as the pipeline's input or output schema.
Source-to-target mapping	Which source column (or expression) fills each target column, with type + null rules.
Transformation node	A step in the DAG (filter, join, derive, …). Each has an input and an inferred output schema.
Join / filter / aggregate / expression / cast / lookup	Relate two inputs / keep rows / group+measure / compute a column / change type / enrich from another table (lookup = gap).
Data lineage	Which source fields feed a target column, through which transforms.
Full vs incremental load	Full = reload everything (the only load strategy today); incremental = deltas (gap).
CDC	Change Data Capture — stream source changes (gap; needs execution).
Pipeline validation	Structural + schema + transform checks before you trust the design.
SQL generation	Emitting runnable SQL text from the design (DaMod does this; it does not run it).
Execution runtime	The engine that actually moves data (external today).
Data quality / reconciliation	Null/duplicate/type checks and source↔target row/total comparisons.

■ NOTE
Three distinct activities this handbook keeps separate: **designing** a pipeline, **generating** an artifact (SQL/DDL), and **executing** it. DaMod does the first two; execution is external.

3. DaMod architecture

The diagram shows the verified relationships. Green = ETL engines available today; blue = existing modeling/connector foundation; grey = not implemented (shown only to place the gaps).



3.1 Available today

- Modeling engine, connectors (metadata discovery), snapshots/drift.
- Pipeline spec v2 (flat objects + transform DAG), persisted deterministically to `pipelines.json`.
- Schema propagation, validation, SQL + target-DDL generation, and the ETL Designer canvas.

3.2 Not implemented (do not treat as working)

- Execution runtime (no data movement — connectors are metadata-only).
- Copilot ETL actions (the general modeling Copilot is not pipeline-aware).

4. Environment setup and prerequisites

4.1 Launch DaMod

1. From the repo root: `source ~/.cargo/env` then `npm install` (first time) and `npm run tauri dev`.
2. Expected: Vite ready on http://localhost:1420, cargo “Finished”, a native window opens.
3. Create or open a **workspace folder** from the Welcome screen (a workspace is a folder of JSON files).

✓ RUNNABLE IN DAMOD TODAY
Launch, workspace, modeling, Import SQL, pipeline design, and SQL/DDL generation all run in DaMod with no database required.

4.2 Optional disposable Postgres (only for external execution, Ch.13/14)

```
brew services start postgresql@16      # or Docker
createdb damod_etl_test
```

■ LIMITATION
Never use production credentials or datasets. Use a throwaway database; DaMod never connects to run the pipeline — you run the generated SQL yourself.

4.3 Reproducible state

Keep the synthetic fixtures in `docs/etl_e2e/fixtures/` (source_schema.sql, source_data.sql, target_schema.expected.sql, verify.sql). Work in a fresh workspace folder so the test never touches other models.

5. End-to-end business use case

Retail Sales Analytics Pipeline. Integrate customer, product, order, and order-item data into an analytics-ready sales model that answers: total sales by product, top customers, sales by date, units sold, and average order value.

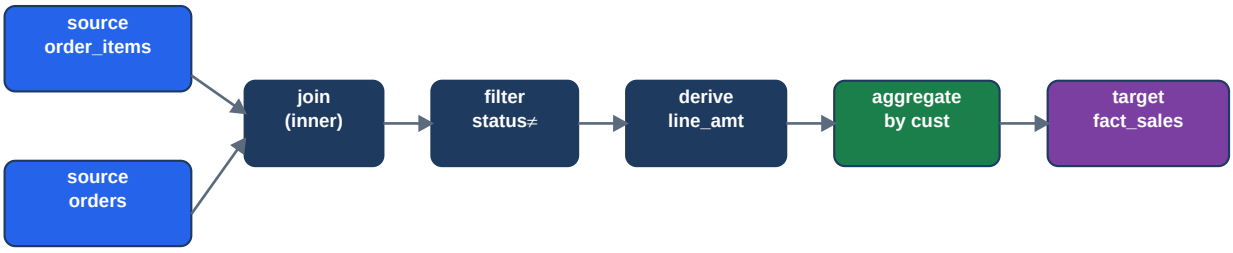
5.1 Source → target

Source entities	Target entities	Grain
customers, products, orders, order_items	dim_customer, dim_product, fact_sales	fact_sales = one row per non-cancelled order_item (line grain)

5.2 Transform rules

- Join order_items to orders (inner) to attach customer + order date.
- Filter out cancelled orders.
- Derive `line_amount = quantity * unit_price`.
- Optionally aggregate to sales-by-customer/day (aggregate node) — or keep line grain.
- Dimensions are straight column selects (customers → dim_customer, products → dim_product).

5.3 Data-flow (fact_sales)



■ LIMITATION
Simplification vs the ideal: lookup/SCD nodes are not implemented, and key-based dedup is not yet available. The use case stays fully testable at line grain; aggregation is optional.

6. Source dataset and schema

Deterministic synthetic data (full files in `docs/etl_e2e/fixtures/`). Edge cases: a NULL email (customer 4), a 0.00 price (product 30), a cancelled order (102), and a zero-amount line (item 1006).

6.1 Schema (source_schema.sql)

```
CREATE TABLE customers (
  customer_id integer PRIMARY KEY, name varchar(100) NOT NULL,
  email varchar(200), country varchar(2) NOT NULL);
CREATE TABLE products (
  product_id integer PRIMARY KEY, sku varchar(20) NOT NULL, name varchar(100) NOT NULL,
  unit_price numeric(12,2) NOT NULL, category varchar(40));
CREATE TABLE orders (
  order_id integer PRIMARY KEY,
  customer_id integer NOT NULL REFERENCES customers(customer_id),
  order_date date NOT NULL, status varchar(20) NOT NULL);
CREATE TABLE order_items (
  order_item_id integer PRIMARY KEY,
  order_id integer NOT NULL REFERENCES orders(order_id),
  product_id integer NOT NULL REFERENCES products(product_id),
  quantity integer NOT NULL, unit_price numeric(12,2) NOT NULL);
```

6.2 Sample records (counts: customers 4, products 3, orders 5, order_items 7)

order_item	order	status	cust	product	qty	unit_price	line_amount
1000	100	shipped	1	10	2	9.99	19.98
1001	100	shipped	1	20	1	19.50	19.50
1002	101	shipped	2	10	5	9.99	49.95
1003	102	cancelled	3	30	1	0.00	excluded
1004	103	shipped	1	20	3	19.50	58.50
1005	104	pending	4	10	1	9.99	9.99
1006	103	shipped	1	30	10	0.00	0.00

7. Source model creation

✓ RUNNABLE IN DAMOD TODAY
Offline path: reverse-engineer the schema from SQL — no database needed.

1. Open/create a workspace folder.
2. **File ribbon** → **Import SQL** (opens “Reverse-engineer from SQL”).
3. “Load file…” → fixtures/source_schema.sql (or paste it); set model name `retail_source`.
4. Confirm the live preview shows **4 entities, 3 relationships**; click **Import**.
5. In Model Explorer, expand entities; verify PKs (*_id), FK relationships, and types (order_items.quantity integer, unit_price numeric(12,2); customers.email nullable).
6. Validation runs live; save (autosave) and reopen to confirm persistence.

■ NOTE
This imports **metadata** (structure), not data. Loading rows into a database (Ch.13) is a separate, external step.

8. Target model design

Two supported paths for the target (dim_customer, dim_product, fact_sales):

- **Fast:** File ribbon → Import SQL → fixtures/target_schema.expected.sql → name `retail_target`.
- **Manual:** New Model dialog (target DB = Postgres) → add entities/columns/keys by hand.

Why this target answers the business questions: **dim_customer**/**dim_product** provide the slice-by attributes; **fact_sales** at line grain carries quantity, unit_price, and the derived `line_amount`, so totals/averages/units roll up by product, customer, or date.

You may also reuse domains and apply Naming/Data-Type Standards to the target (Home ribbon → Domains / Naming / Types). Optional for this pipeline.

> ■ **NOTE**
> You don't have to hand-build the target: DaMod can generate target DDL from the pipeline (Ch.13), deriving columns from the DAG's target node.

9. Source-to-target mapping

The mapping is the test oracle. Configure column edits in the **Mapping** tab (Data ribbon → Pipelines) and the multi-table fact in the visual **Designer** (Ch.11). Full table in Appendix B.

9.1 fact_sales (line grain)

Target column	Source	Transform	Type	Req?
order_item_id	order_items.order_item_id	identity	integer	yes
order_id	order_items.order_id	identity	integer	yes
customer_id	orders.customer_id (via join)	identity	integer	yes
product_id	order_items.product_id	identity	integer	yes
order_date	orders.order_date (via join)	identity	date	yes
quantity	order_items.quantity	identity	integer	yes
unit_price	order_items.unit_price	identity	numeric(12,2)	yes
line_amount	quantity, unit_price	derive: quantity*unit_price	numeric(12,2)	yes

Filter on the joined orders: `status != 'cancelled'` (excludes order 102).

■ NOTE
This table is the expected behavior / oracle. The implemented path is the Designer DAG + the Mapping tab for flat column edits — not a one-click auto-map across joined tables.

10. Transformation design

Transform	Purpose	In→Out schema	Implemented?
select	Keep/rename columns	subset + renames	Yes (node)
rename/alias	Rename a column	via select renames	Yes
filter	Keep matching rows (8 ops incl. null/in)	passthrough	Yes (node)
derive	Expression column (e.g. qty*price)	input + new col	Yes (node)
cast	Change a column's type	retype a column	Yes (node)
join	Relate two inputs (inner/left) on keys	left + right cols	Yes (node, L/R ports)
aggregate	group-by + sum/count/count_distinct/avg/min/max	groups + measures	Yes (node)
distinct	Full-row de-duplication	passthrough	Yes (node)
null handling	COALESCE / default via expression + filter	via derive/filter	Workaround (no node)
lookup	Enrich from another table (non-join)	add cols	No (gap)
data-quality checks	Null/duplicate/type rules	n/a	Partial (validation only)

Every node's output schema is inferred (names + types) and flows downstream; the right panel shows it live. Derived/aggregate columns carry explicit types (e.g. sum→decimal, count→bigint).

■ LIMITATION
Expressions (derive, filter values) are passed through to SQL verbatim — they must be valid for the source dialect. DaMod validates referenced columns, not full expression syntax.

11. Build the pipeline step by step

1. **Data ribbon** → **Pipelines** → **New pipeline**: source model retail_source, name `retail_sales`, target platform postgres, select all 4 objects → Plan full load & save. (Born as spec v2 with a starter graph.)
2. **Data ribbon** → **Designer** → select retail_sales. If no graph, click **Create starter graph**.
3. Use the left **palette** to add nodes; drag to position; drag between nodes to connect; Delete removes a selection. Configure each node in the right **NodeInspector**.
4. fact_sales DAG: source order_items + source orders → **join** (inner; key order_id=order_id; wire the two sources to the join's left then right ports) → **filter** (status != cancelled) → **derive** (line_amount = quantity*unit_price, decimal) → **select** (keep the 8 fact columns) → **target** (targetObject = fact_sales).
5. Optional: insert an **aggregate** before target (group by customer_id, SUM(line_amount) AS total_sales).
6. dim_customer / dim_product: source → (optional select) → target.
7. Watch the bottom **diagnostics** panel: resolve errors (warnings are OK). Select the derive node to confirm line_amount appears in its output schema.
8. Save; reopen the workspace and the Designer to confirm the graph persists.

> **■ BLOCKED / PRODUCT GAP**
> Pipeline-edit **undo/redo** is not available (edits persist immediately; model ■Z does not affect pipelines). Re-edit manually to revert. Tracked as gap G-UNDO.

12. Validation and data quality

Validation has distinct levels. Errors should block trust/export; warnings are advisory.

Level	Checks	Example message
Structural	node ids, edges, no dangling, no cycle, input arity	“Connection would create a cycle”; “Join needs exactly two inputs”
Schema	source/target columns exist, required mapped, dup outputs	“Required target column … has no source mapping”
Transform	filter/cast/join columns exist, filter value present, derive name unique	“Filter on unknown column”; “duplicates an existing column”
Capability	source/target/runtime support (metadata-only today)	(execution capabilities absent — see Ch.13)
Execution-readiness	credentials, target exists, privileges	N/A in DaMod (external)

12.1 Negative cases to exercise

Trigger	Expected	Blocking?
Rename a source column away (drift)	“Source column … is missing (schema drift?)”	yes
Required target column with no source/transform	“… has no source mapping”	yes
Filter on a non-existent column	“Filter on unknown column …”	yes
String→numeric with no transform	type-conversion warning	no
Connect a node back to an ancestor	connect rejected: “… create a cycle”	yes
Join wired with 1 or 3 inputs	“Join node needs exactly two inputs”	yes
Two sources share a column into a join	duplicate-column warning	no

13. SQL generation and execution

13.1 Generate (runnable in DaMod)

- **SQL**: Designer → SQL toggle, or Pipelines → SQL tab. For a DAG it emits a `WITH`-chain (one CTE per node) per target, in the source dialect.
- **Target DDL**: Pipelines → Target DDL tab (derived from the DAG target node when present).
- **Export**: writes `<name>.extract.sql`, `.target.sql`, and `.pipeline-report.txt`.

Representative generated SQL for fact_sales (shape; exact text comes from the app):

```
WITH "order_items" AS (SELECT * FROM "order_items"),
     "orders"      AS (SELECT * FROM "orders"),
     "join"   AS (SELECT "order_items".*, "orders".* FROM "order_items"
                  INNER JOIN "orders" ON "order_items"."order_id" = "orders"."order_id"),
     "filter" AS (SELECT * FROM "join" WHERE "status" != 'cancelled'),
     "derive" AS (SELECT *, "quantity" * "unit_price" AS "line_amount" FROM "filter"),
     "select" AS (SELECT "order_item_id","order_id","customer_id","product_id",
                         "order_date","quantity","unit_price","line_amount" FROM "derive"),
     "fact_sales" AS (SELECT * FROM "select")
SELECT * FROM "fact_sales";
```

13.2 Execution

■ BLOCKED / PRODUCT GAP
DaMod does NOT execute pipelines. Connectors are metadata-only; the SQL is design-time text. A generated SQL script is not a running ETL job (gap G-EXEC).

13.3 Safe external execution (Category B)

```
psql damod_etl_test -f fixtures/source_schema.sql
psql damod_etl_test -f fixtures/source_data.sql
psql damod_etl_test -f fixtures/target_schema.expected.sql
psql damod_etl_test -c "INSERT INTO dim_customer SELECT * FROM customers;"
psql damod_etl_test -c "INSERT INTO dim_product  SELECT * FROM products;"
psql damod_etl_test -c "INSERT INTO fact_sales <paste the fact_sales WITH-chain SELECT>;"
```

■ LIMITATION
Re-running without TRUNCATE causes a PK violation (no upsert/merge). Truncate between runs. There is no retry/rollback because DaMod does not run the job.

14. Verify the final output

All values are hand-computed from the synthetic data (Appendix C). Run `fixtures/verify.sql` after loading the targets.

Check	Expected
dim_customer rows	4
dim_product rows	3
fact_sales rows	6
SUM(line_amount)	157.92
Cancelled orders in fact	0
fact_sales PK duplicates	0
line_amount = round(qty*price,2) mismatches	0
Sales by customer	cust 1 = 97.98, cust 2 = 49.95, cust 4 = 9.99 (cust 3 absent)
dim_customer #4 email	NULL (null handling preserved)

```
SELECT COUNT(*) FROM fact_sales;                                -- 6
SELECT ROUND(SUM(line_amount),2) FROM fact_sales;              -- 157.92
SELECT COUNT(*) - COUNT(DISTINCT order_item_id) FROM fact_sales;   -- 0
SELECT COUNT(*) FROM fact_sales
 WHERE line_amount <> ROUND(quantity*unit_price,2);            -- 0
```

15. Save, reopen, and maintain pipelines

Capability	Status / how to verify
Persistence	Pipelines saved to pipelines.json (deterministic). Reopen workspace → graph intact.
Reopen / edit	Designer reloads nodes/edges/config/layout.
Undo / redo	NOT supported for pipeline edits (G-UNDO).
Source schema change	Revalidate → drift errors; mappings are not silently rewritten.
Stale metadata	Flagged by validation (missing object/column; unknown source).
Version compatibility	v1 pipelines.json upgrades to v2 on load and gains a starter graph.
Restart recovery	Saved pipelines reload; unsaved in-memory graph edits since last save are lost.

16. Troubleshooting guide

Symptom	Likely cause	Resolution	Status
Import SQL parse error	Unsupported DDL construct	Keep to CREATE TABLE/PK/FK/standard types	By design
Live connection fails	Only metadata discovery exists; creds in keychain	Use Import SQL (offline) for this guide	By design
“Source object … no longer in the model”	Schema drift after a model edit	Re-point the source or fix the model; revalidate	Expected
Join shows “needs two inputs”	Only one edge wired to the join	Connect both sources to left + right ports	Expected
Type-conversion warning	Source/target types differ, no transform	Add a cast node or a transform	Expected
SQL tab is blank	Source model not open	Open the source model so schemas resolve	Expected
Target DDL misses derived cols	Flat objects used, not the DAG	Ensure a target node exists + model open (graph DDL)	Fixed
Can't undo a pipeline edit	No pipeline undo stack	Re-edit manually	Gap (G-UNDO)
Pipeline won't run in app	No execution runtime	Run generated SQL externally	Gap (G-EXEC)

17. Test plan and acceptance criteria

Unique IDs; fill Actual/Pass-Fail when executing. Full negative matrix in Chapter 12 and docs/etl_e2e/ETL_TEST_CASES.md.

ID	Objective	Expected	Actual	P/F
T1	Reverse-engineer source	4 entities, 3 FKs		
T2	Create pipeline + starter graph	Pipeline saved; canvas nodes		
T3	Build fact_sales DAG	0 errors; line_amount in derive schema		
T4	Generate graph SQL	WITH-chain with JOIN+WHERE+derive		
T5	Generate target DDL	CREATE TABLE fact_sales incl. line_amount		
T6	Negative: cycle	connect rejected		
T7	Negative: drift	“source column missing” error		
T8	Save / reopen	graph intact		
T9	External execute + verify	fact=6, total=157.92, checks 0		
T10	Aggregate variant	GROUP BY emitted; totals reconcile		

17.1 End-to-end acceptance checklist

- Source + target models created and validated.
- fact_sales DAG validates with 0 errors; dims mapped.
- Graph SQL + target DDL generated and inspected.
- External run of the generated SQL reproduces the Chapter-14 oracle exactly.
- Save/reopen preserves the pipeline.

18. Current limitations and roadmap

Area	State	Note
Source/filter/select/derive/cast/join nodes	Done	Schema-propagated + SQL
Aggregate node	Done	group-by + sum/count/count_distinct/avg/min/max (G-AGG)
Distinct (dedup) node	Done	full-row DISTINCT (G-XFORM partial)
Graph target DDL	Done	from DAG target node (G-DDL-GRAPH)
Union / lookup / SCD / key-based dedup	Gap	G-XFORM remaining
Pipeline undo/redo	Gap	G-UNDO
Incremental / CDC / scheduling	Gap	G-INC (needs execution)
Copilot ETL actions	Gap	G-AI
Native execution	Gap	G-EXEC — the headline gap (external today)

Recommended sequence: G-XFORM (union, then lookup) → G-UNDO → G-AI (read-only compliance/suggestions first) → G-INC (design-time) → G-EXEC (a dry-run-first, explicitly confirmed Rust write adapter). G-EXEC carries the most risk (real data movement) and should be gated and isolated.

Appendix A — Quick-start checklist

1. npm run tauri dev; open a fresh workspace folder.
2. File ribbon → Import SQL → source_schema.sql (retail_source).
3. File ribbon → Import SQL → target_schema.expected.sql (retail_target).
4. Data ribbon → Pipelines → New (retail_source, postgres, all objects) → save.
5. Data ribbon → Designer → Create starter graph → build the fact_sales DAG (Ch.11).
6. Check diagnostics = 0 errors; SQL toggle to review generated SQL; Target DDL tab.
7. Export report + SQL + DDL.
8. (External) load fixtures into a disposable Postgres, run the SQL, run verify.sql.
9. Confirm fact_sales = 6 rows, SUM(line_amount) = 157.92.

Appendix B — Mapping reference

dim_customer

Target	Source	Transform	Null
customer_id	customers.customer_id	identity	n/a
name	customers.name	identity	NOT NULL
email	customers.email	identity	passes NULL (cust 4)
country	customers.country	identity	NOT NULL

dim_product

Target	Source	Transform
product_id	products.product_id	identity
sku	products.sku	identity
name	products.name	identity
unit_price	products.unit_price	identity (0.00 kept)
category	products.category	identity

fact_sales mapping is in Chapter 9.1.

Appendix C — Test data and expected results

Derived fact_sales (6 rows; cancelled order 102 excluded)

item	cust	product	qty	unit_price	line_amount
1000	1	10	2	9.99	19.98
1001	1	20	1	19.50	19.50
1002	2	10	5	9.99	49.95
1004	1	20	3	19.50	58.50
1005	4	10	1	9.99	9.99
1006	1	30	10	0.00	0.00

Grand total = 19.98+19.50+49.95+58.50+9.99+0.00 = **157.92**. By customer: 1=97.98, 2=49.95, 4=9.99 (sum 157.92 ✓). dim_customer=4, dim_product=3.

Appendix D — Glossary and references

Term	Definition
DAG	Directed acyclic graph of transform nodes; DaMod rejects cycles.
Grain	The level of detail of a fact table's rows (here: one per order_item).
Line amount	quantity × unit_price for an order line.
Metadata-only connector	Reads schema structure, never table rows.
Design-time	Building/validating/generating without executing.

Repository references

- Playbook + fixtures: `docs/etl_e2e/` (ETL_E2E_WORKFLOW_PLAYBOOK.md, ETL_*_*.md, fixtures/*.sql).
- Pipeline engine: `src/connectors/pipeline/` (spec.ts, planner.ts, graph.ts, graphSql.ts).
- Designer UI: `src/etl/` (EtlDesigner.tsx, PipelineCanvas.tsx, NodeInspector.tsx).
- Decision log: `DECISIONS.md`; connector contract: `docs/CONNECTORS.md`.